SUSTech CS305 Fall 2025 Project

> P2P File Transfer with Reliable Data Transfer
>
> GitHub starter repository: https://github.com/OctCarp/sustech-cs305-f25-project-starter

1. Introduction

The documentation introduces the core logic of the entire project. The documentation may be a bit long, but it is written this way to explain the details clearly. Starting early will help you identify and solve potential issues sooner. For the specific code implementation, we will provide **a Setup tutorial** and simple Example code to help you familiarize yourself with the code framework. We recommend that you first gain a general understanding of the project background, and then combine that understanding with the code to comprehend the project's logic.

In this project, you are required to build a **reliable peer-to-peer (P2P) file transfer application with congestion control**. Note that this project uses **UDP** as the transport layer protocol. **All requirements are implemented at the application layer**.

If you have any questions or uncertainties related to project framework, you can raise an issue . We may also update documents or files to address important issues or release new files. You can check the repository for the latest updates.

2. Overview

2.1 Our P2P File Transfer Core Concepts

File: A file is a collection of data stored on a computer, whose size is measured in bytes.

Chunk: The unit for downloading. A file is divided into a set of equal-sized chunks. The size of each chunk is 512KiB. To distinguish these chunks, **a cryptographic hash (e.g., SHA-1) with a fixed size of 20 bytes is calculated** for each of them. Each chunk can be uniquely identified by its hash value.

Peer: A peer is a program running with a fixed hostname and port. Initially, each peer holds multiple chunks, which are not necessarily contiguous. We call the set of chunks owned by a peer a **fragment** of the file. Note that the fragments held by different peers may be different and may overlap.

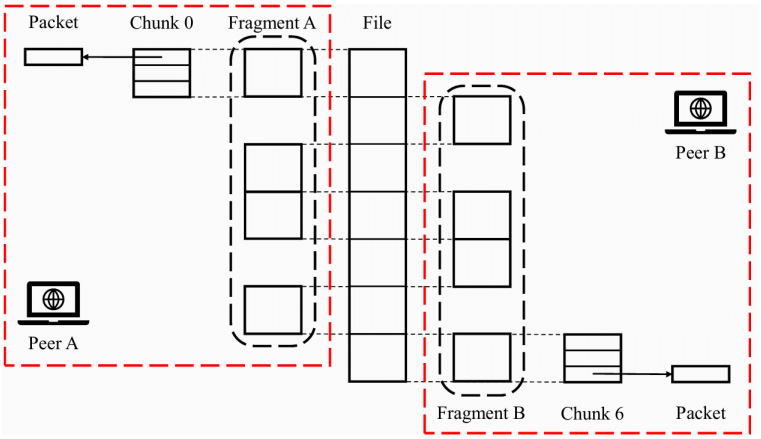
For information on the definition and implementation of our P2P concepts, see the "*P2P Key File Definitions*" subsection in the Setup tutorial.

2.2 P2P Transmission Overview

This system consists of a single file distributed among multiple peers. Initially, each peer holds a subset of the file's chunks, and the combined set of all initial chunks is guaranteed to form the complete file.

Upon receiving a `DOWNLOAD` command from `stdin`, a peer must download all file chunks it does not already possess. To begin, the peer identifies its missing chunks by checking its local fragment and then requests those chunks from other peers.

Downloading a single chunk involves a handshaking process followed by data transfer. Each chunk must be divided into multiple packets for transmission. A peer can download different chunks from multiple peers concurrently.



The goal for each peer is to acquire the complete set of file chunks. **Once a peer has received all requested chunks, it collects them into a dictionary, using the chunk hashes as keys. Finally, it serializes this dictionary into a binary file using `pickle`, saving it under the filename specified in the `DOWNLOAD` command.**

For information on how to build fragments and hashes from files, see the "*Prepare chunk files*" subsection in the Setup tutorial.

For file transfer details, see Example code.

2.3 Packet Structure

Type Code (1 byte)	Header Length (1 byte)	Packet Length (2 byte)
Sequence Number (4 byte)		
ACK Number (4 byte)		
Payload		

A specific packet format is defined with a maximum length of 1400 bytes.

2.3.1 Header Fields

- Type Code: indicate the packet type
- Header Length (Bytes) / Packet Length (Bytes)
- Sequence / ACK Number: Plays a similar role to the Sequence and Acknowledgment numbers in TCP.
    - **Note:** For simplicity, the Seq\ACK numbers here are counted **per packet**, rather than per byte as in standard TCP.
- Payload: All parts other than the header mentioned above are in the payload (e.g. chunkdata).

2.3.2 Packet Types

- `WHOHAS` **(0):** To ask which peers have certain chunks.
- `IHAVE` **(1):** The response to a `WHOHAS` query.
- `GET` **(2):** To request a specific chunk.
- `DATA` **(3):** To transfer the actual chunk data.
- `ACK` **(4):** To acknowledge received data packets.
- `DENIED` **(5):** The denial to a `WHOHAS` query (e.g., if connection limits are reached).

For implementation details on how to specifically build and parse these packets in Python, see the "*Creating Packets in Python*" subsection of the Setup tutorial and Example code.

3. Implementation

- **Programming language**: This project base developed using **Python 3.12**, but lower versions **(at least 3.10)** may also be able to run. If you find any compatibility issues caused by Python version, you can submit an issue to us.
- **Library**: In your peer source code, you cannot use any library other than Python Standard Library and `matplotlib`. In test script, you may be allowed to use `pytest` , `networkx` etc..
- **Linux**: this project requires you to configure a Linux system. You can use native Linux, Windows Subsystem for Linux (WSL), virtual machines, Docker, and other implementations. Just make sure you can run the Project code normally. You can refer the "*Linux Setup*" section in the Setup tutorial.
- **Single-thread only**: Single-thread Concurrency: You are required to implement this project in a single thread. The use of `multithreading`, `multiprocessing`, or `asyncio` is prohibited. Concurrency should be achieved using non-blocking I/O like the `select` module, see Example code.

4. Basic Requirements

4.1 P2P File Transfer

To acquire missing chunks, a peer initiates a discovery and request sequence:

1. **WHOHAS:** The downloading peer broadcasts a `WHOHAS` packet to all known peers, containing the list of hashes for the chunks it needs.
2. **IHAVE & DENIED:** Peers that receive the `WHOHAS` check their local storage. If they possess any requested chunks and are below their concurrent upload limit (`max_send`), they reply with an `IHAVE` packet listing the available chunk hashes. Otherwise, they respond with a `DENIED` packet.
3. **GET:** Upon receiving `IHAVE` replies, the downloading peer selects a source peer for each needed chunk. It then sends a `GET` packet to a specific peer for each specific chunk hash.

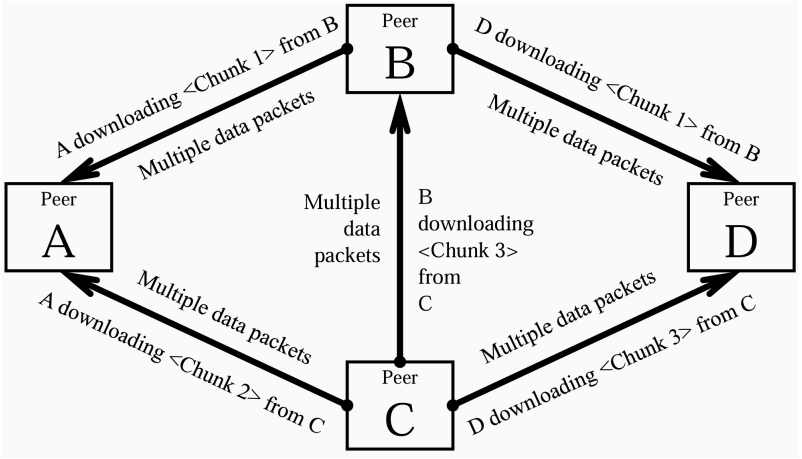
Note that all data except for the header is is transferred in the packet payload.

For a detailed walkthrough of this process, see the "*Handshaking*" subsection in the Setup tutorial and Example code.

4.2 Chunk Transfer

A single peer-to-peer session (e.g., Peer A downloading from Peer B) can only transfer one chunk at a time. Peer A cannot request a new chunk from Peer B until the transfer of the current chunk is complete. However, a peer can concurrently download different chunks from different peers (e.g., A from B, and A from C).

For example in figure below, peer A receives different chunks from B and C at the same time. Peer B sends chunk data to A and D and receives data from C at the same time.



4.3 Reliable Data Transfer (RDT)

For concepts of RDT, You can read Section 3 of *A Top Down Approach* for details.

After a successful handshake, RDT for chunk data should be achieved through a TCP-like mechanism using sequence numbers, acknowledgements (ACKs), and retransmissions.

4.3.1 Core Mechanisms

1. **Acknowledgements:** The receiver must send an `ACK` packet for every `DATA` packet received, confirming its sequence number.
2. **Timeout & Retransmission:**
    - The sender starts a timer after sending a `DATA` packet.
    - If the corresponding `ACK` is not received within a dynamically calculated `TimeoutInterval`, the packet is retransmitted.
3. **Fast Retransmit:**
    - Upon receiving **3 duplicate ACKs** for the same sequence number, the sender immediately retransmits the corresponding `DATA` packet without waiting for the timeout.
    - This mechanism is triggered at most once per packet transmission attempt.
    - Please note, for simplicity, the duplicate ACK is per-round, that is, each seq-ack round has its own duplicate ACK counter. The counter should not be reset to 0 after this retransmission, thus preventing further retransmissions for the same packet (e.g., when the count reaches 6). So, when you first trigger retransmission when duplicate ACK==3, it will not be triggered again when duplicate ACK==6.

4.3.2 Timeout Calculation

As introduced in lecture, you need to estimate the RTT for determining the timeout interval. You could refer the RTT formula given in Section 3.5.3 of the textbook. **In your project**, to compute `Timeout` using:

- $\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$
- $\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$
- $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$

where $\alpha = 0.15, \beta = 0.3$

4.4 Congestion Control

You need to implement a congestion control mechanism similar to TCP to dynamically adjust the send window (`cwnd`). Note: `cwnd` is expressed in packets, not bytes.

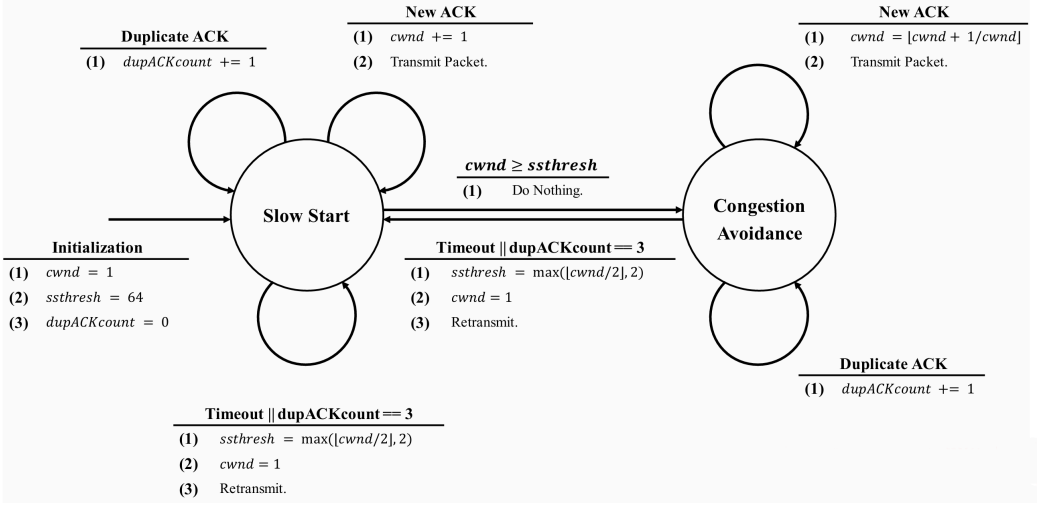
4.4.1 Slow Start

At the initial connection, `cwnd` is set to 1 , `ssthresh = 64`. With each `ACK` received, `cwnd` increases by 1.

4.4.2 Congestion Avoidance

After `cwnd` reaches the slow start threshold `ssthresh`, `cwnd` increases the window size by $\frac{1}{\text{cwnd}}$ packet upon receiving ACK. Since the packets sent each time must be an integer, you should use $\lfloor \text{cwnd} \rfloor$.

Similar to the "Slow Start", **if there is a loss in the network (resulting from either a time out or duplicate ACKs)**, `ssthresh` is set to $\max(\lfloor \text{cwnd}/2 \rfloor, 2)$. The `cwnd` is then set to 1 and the system will jump to the "Slow Start" state again.



5. Evaluation and Grading

The project will be evaluated through 3 types of tests: Basic Tests, Comprehensive Tests, and Presentation. The **maximum score is 100** points, with an additional **10 bonus** points available. **Testing scripts will be released at 2 checkpoints to help track progress.**

To understand how to run peers with the network simulator for testing, please refer to the "*Setup Network Simulator*", "*Example Process*" and "*Run `pytest` Script*" sections in the Setup tutorial, and see our released test cases and scripts.

5.1 Grading Breakdown

5.1.1 Basic Tests (60 points)

- *Handshaking (10 pts):* Evaluates correct flooding of WHOHAS and proper handshake completion using WHOHAS, IHAVE, and GET packets.
- *Reliable Data Transfer (10 pts):* Verifies reliable data transfer under packet loss conditions.
- *Congestion Control (20 pts):* Assesses the congestion control algorithm. A plot demonstrating window size changes (e.g., slow start, congestion avoidance, reaction to loss) is required for evaluation.
- *Concurrency (10 pts):* Checks the ability to download concurrently from multiple peers and send DENIED when connection limits are reached.
- *Robustness (10 pts):* Tests handling of corner cases like peer crashes and severe packet loss. Assume that if a peer crashes, it will never restart.

5.1.2 Comprehensive Tests (40 points)

- *Complex Cases (30 points):* Performance is evaluated on several different network topology tasks. Tests involve relatively larger and more complex topologies and may include peer crashes, requiring a robust implementation.
    - Some of the tests are public, while others are hidden. Hidden ones are tested and graded by TAs after you submit your code.
    - We encourage you to design and construct your own test cases (e.g., custom topologies, simulator settings) to perform more complex testing. During the presentation(see below), you could use these custom tests to demonstrate the robustness of your implementation and explain any key performance improvements you made.
- *Performance ranking (10 points):* We will test performance on the unified platform after your final submission. We will design some samples, and and test the completion time (speed) of your code, grading is based on a ranking ratio (**for example**, top 10% 10 points, top 40% 6 points, ...). Specific grading plan is still under discussion (we may provide an plan based on the final distribution of results). This part serves as an **advanced requirement** — please focus primarily on the correctness of other requirements.

Note: For parts that are not explicitly defined (such as whether to choose GBN or SR for the fast retransmission mechanism), please choose a reasonable method to implement it and briefly introduce it in the presentation. This can also be your performance highlight.

If you use a different implementation than the one in the documentation (such as the calculation of the RTT and timeout), please clearly indicate this in your presentation and ensure fairness in the performance comparison (e.g., adopt a good general design instead of directly adjusting appropriate parameters based on the test cases).

5.1.3 Presentation (10 points)

- In your presentation, you need to show your congestion control diagram.
- It should also include a brief overview of your implementation approach, your system robustness (if any), and any key performance improvements (if applicable).
- Q&A Session

Please refer to the next section for more details.

6. Submission and Presentation

Code Submission: **23:59 at Dec. 19th, 2025 (Fri. in Week 15)**

- As we mentioned before, we will release the test scripts in 2 parts. You will receive points for passing these test scripts. Finally, you will need to submit your code to Blackboard.
- You only need to submit all the code in the `src/` directory. Compress your `src` folder into `gxx_src.zip` (`xx` is your group ID, e.g. `g03_src.zip`). We will grade your project based on this code.

During the Week 16 lab session, you will need to give a presentation.

- Besides related content mentioned above, you should mainly describe your design **highlight** (that is, implementations similar to those in the documentation and framework like normal handshaking **need no further explanation**. However, if you don't have any additional highlights, you can just describe your basic implementation.
- We will also ask questions related to the project to check how familiar you are with these details. If you answer poorly, we may assume you relied too much on external tools to finish the project, which will affect your overall grade. But don't worry. As long as you've done the project work yourself, this part shouldn't be difficult for you.
- You should submit your presentation slides (as a PowerPoint or PDF (preferred) file) like `gxx_pre.pdf`. You will not be required to submit a report.

7. Collaboration Policy

This project must be completed independently by each group.

You may discuss general ideas with classmates, but sharing or copying code is strictly prohibited.